SENAI

PROGRAMAÇÃO BACK-END 2

JavaScript moderno e programação assíncrona

Coleções, Promises, async/await e concorrência

Projeto api-produtos | Aula 02

Missão da aula

Experimentar no Sandbox e aplicar logo depois à api-produtos

- Transformar coleções sem alterar a entrada
- Escolher const e let com intenção
- Usar template strings, desestruturação e spread
- Tratar Promises com async/await
- Comparar sequência com Promise.all

Resultados esperados

- Usar map, filter e find conforme o resultado desejado
- Criar novas versões de objetos com spread
- Reconhecer os estados de uma Promise
- Tratar falhas previstas com try/catch
- Coordenar operações independentes de forma concorrente

Mapa da aula

- Fundamentos assíncronos
- JavaScript moderno e coleções
- Promises e async/await
- Sequência e concorrência
- Evolução da api-produtos
- Atividades e fechamento

Fundamentos essenciais

F

Entender o resultado futuro antes de escrever o código que o consome.

Uma Promise muda de estado

O estado final não volta a ser pendente.



Async/await organiza o consumo

Sem await

- A função continua
- O resultado chega depois
- then/catch organizam o fluxo

Com await

- A função atual aguarda
- O Event Loop continua ativo
- try/catch trata a falha

Concorrência não é paralelismo

Concorrência de I/O

- Várias esperas em andamento
- Disco, banco ou rede
- Uma tarefa avança enquanto outra espera

Paralelismo de CPU

- Cálculos simultâneos
- Diferentes núcleos ou threads
- Exige mecanismos específicos

Quando Promise.all ajuda

Independentes

Nenhuma operação precisa do resultado da outra

Iniciadas juntas

As Promises são criadas antes da espera

Falha conjunta

Uma rejeição faz o Promise.all rejeitar

Mini-desafio 1

01

Descobrir o que const impede e criar uma cópia com spread.

Teste reatribuição e mutação

```
node -e "const valor = 1; valor = 2"
node -e "const item = { estoque: 1 }; item.estoque = 2; console.log(item)"
```

O que const realmente impede?

Responda antes de executar ou avançar

Por que somente a primeira linha falha?

A) const congela qualquer objeto

B) const impede reatribuir o identificador

C) const transforma propriedades em texto

Crie o arquivo (copias.js)

```
const original = { id: 1, nome: 'Mouse', estoque: 5 };
const atualizado = { ...original, estoque: 8 };

console.log({ original, atualizado });
```

Cópia rasa com spread

original

- Mantém estoque 5
- Não recebe reatribuição
- Continua sendo objeto

atualizado

- É um novo objeto
- Recebe estoque 8
- Copia propriedades do primeiro nível

Mini-desafio 2

02

Extrair propriedades e montar texto com expressões.

Acrescente a copias.js

```
const { nome, estoque } = atualizado;
console.log(`${nome} possui ${estoque} unidades.`);
```

Duas ferramentas, duas funções

Desestruturação

Extrai valores
de propriedades
ou posições

Template string

Intercala texto
e expressões
com ${...}

Propriedade ausente

Sem valor padrão,
o resultado é
undefined

Mini-desafio 3

SENAI

03

Escolher map, filter ou find pelo formato do resultado.

Crie o arquivo (colecoes.js)

```
const produtos = [
  { id: 1, nome: 'Mouse', preco: 80, categoria: 'Periféricos' },
  { id: 2, nome: 'Monitor', preco: 900, categoria: 'Vídeo' },
  { id: 3, nome: 'Teclado', preco: 120, categoria: 'Periféricos' },
];
```

Aplique os três métodos

```
const nomes = produtos.map(({ nome }) => nome);
const perifericos = produtos.filter(
  ({ categoria }) => categoria === 'Periféricos'
);
const monitor = produtos.find(({ id }) => id === 2);

console.log({ nomes, perifericos, monitor });
```

Map, filter e find

map

Transforma todos
Mantém a quantidade
Devolve array

filter

Seleciona zero ou mais
Pode devolver []
Devolve array

find

Primeira correspondência
Pode devolver undefined
Devolve item

Filtre, transforme e copie

```
const reajustados = produtos
  .filter(({ categoria }) => categoria === 'Periféricos')
  .map((produto) => ({
    ...produto,
    preco: produto.preco * 1.1,
  }));

console.log({ produtos, reajustados });
```

Mini-desafio 4

SENAI

04

Observar uma Promise terminar com valor ou erro.

Crie o arquivo (promessa.js)

```
function consultarProduto(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 1) resolve({ id: 1, nome: 'Mouse' });
      else reject(new Error(`Produto ${id} não encontrado.`));
    }, 80);
  });
}
```

Consuma com then e catch

Execute com ID 1 e depois troque para 99.

```
consultarProduto(1)
  .then((produto) => console.log('Sucesso:', produto))
  .catch((erro) => console.error('Falha:', erro.message));
```

Mini-desafio 5

05

Reescrever o consumo com async, await e try/catch.

Continue: crie executar (promessa.js)

```
async function executar(id) {
  try {
    const produto = await consultarProduto(id);
    console.log('Sucesso:', produto);
  } catch (erro) {
    console.error('Falha:', erro.message);
    process.exitCode = 1;
  }
}

executar(1);
```

O que mudou na escrita

- async faz executar devolver uma Promise
- await aguarda consultarProduto dentro da função
- try organiza o caminho de sucesso
- catch trata a rejeição prevista
- process.exitCode torna a falha observável no terminal

Mini-desafio 6

06

Comparar duas esperas em sequência e iniciadas juntas.

Defina uma espera (tempo.js)

```
const esperar = (ms, valor) =>
  new Promise((resolve) => setTimeout(() => resolve(valor), ms));
```

Antes de executar, abra o package.json e inclua:

```
"type": "module"
```

A partir do Node.js 24, o motor já consegue identificar e rodar módulos ESM nativamente em muitos casos, mas é uma excelente prática declarar isso explicitamente no package.json.

Meça a sequência

```
const inicioSequencial = performance.now();
await esperar(80, 'produto');
await esperar(60, 'categorias');
console.log('Sequencial:',
  Math.round(performance.now() - inicioSequencial));
```

Meça a concorrência

```
const inicioConcorrente = performance.now();
const resultado = await Promise.all([
  esperar(80, 'produto'),
  esperar(60, 'categorias'),
]);
console.log('Concorrente:',
  Math.round(performance.now() - inicioConcorrente), resultado);
```

O que o tempo deve mostrar

Sequencial

- 80 ms e depois 60 ms
- Tempo próximo da soma
- A segunda depende da espera anterior

Concorrente

- As duas esperas começam
- Tempo próximo da maior
- Resultados chegam juntos

Quando usar Promise.all?

Responda antes de executar ou avançar

Qual condição torna a execução concorrente segura?

A) A segunda operação precisa do resultado da primeira

B) As operações são independentes

C) O código contém qualquer função async

Evolução hands-on da api-produtos

07

Evolua o mesmo projeto criado na Aula 01, digitando cada alteração.

Setup: entre na API da raiz

Parta da pasta da Aula 02 e valide a configuração existente

```
cd ..\api-produtos
npm run check
copy .env.example .env
npm start
```

Evolua os scripts (package.json)

Substitua o bloco scripts atual por este

```
"scripts": {
  "start": "node app.js .env 1",
  "diagnostico": "node app.js",
  "check": "node --check app.js",
  "produto-ausente": "node app.js .env 999",
  "id-invalido": "node app.js .env abc"
}
```

app.js | configuração e catálogo

Digite por cima do app.js da Aula 01 | trecho 1 de 6

```
const arquivoDeConfiguracao = process.argv[2];
const idInformado = process.argv[3];
let configuracaoCarregada = true;

if (arquivoDeConfiguracao) {
  try { process.loadEnvFile(arquivoDeConfiguracao); }
  catch {
    console.error(`Arquivo não encontrado: ${arquivoDeConfiguracao}`);
    process.exitCode = 1;
    configuracaoCarregada = false;
  }
}
```

app.js | valide o ambiente

app.js | trecho 2 de 6

```
const obrigatorias = ['PORT', 'NOME_ALUNO', 'TURMA'];
const ausentes = obrigatorias.filter(
  (nome) => !process.env[nome]?.trim()
);

if (configuracaoCarregada && ausentes.length) {
  console.error(`Configure no .env: ${ausentes.join(', ')}`);
  process.exitCode = 1;
  configuracaoCarregada = false;
}
```

app.js | cadastre os produtos

app.js | trecho 3 de 6

```
const produtos = [
  { id: 1, nome: 'Mouse', preco: 80, estoque: 10, categoria: 'Periféricos' },
  { id: 2, nome: 'Monitor', preco: 900, estoque: 3, categoria: 'Vídeo' },
  { id: 3, nome: 'Teclado', preco: 120, estoque: 8, categoria: 'Periféricos' },
];

const esperar = (ms) => new Promise(
  (resolve) => setTimeout(resolve, ms)
);
```

app.js | busque o produto

app.js | trecho 4 de 6

```
async function buscarProdutoPorId(id) {
  await esperar(100);
  if (!Number.isInteger(id)) {
    throw new Error('Identificador inteiro exigido');
  }
  const produto = produtos.find((item) => item.id === id);
  if (!produto) {
    throw new Error(`Produto ${id} não encontrado`);
  }
  return { ...produto };
}
```

app.js | liste as categorias

app.js | trecho 5 de 6

```
async function listarCategorias() {
  await esperar(100);
  const categorias = produtos.map(
    ({ categoria }) => categoria
  );
  return [...new Set(categorias)];
}
```

app.js | execute as consultas

app.js | trecho 6 de 6

```
async function executar() {
  if (!configuracaoCarregada) return;
  try {
    const id = Number(idInformado ?? 1);
    const [produto, categorias] = await Promise.all([
      buscarProdutoPorId(id), listarCategorias()
    ]);
    console.log('Produto:', produto);
    console.log('Valor em estoque:', produto.preco * produto.estoque);
    console.log('Categorias:', categorias);
  } catch (erro) {
    console.error(erro.message);
    process.exitCode = 1;
  }
}
executar();
```

Caminho de sucesso

Produto e categorias começam juntos no Promise.all

```
npm start
```

Confirme a execução simultânea

- O diagnóstico existente permanece válido
- Produto 1 é localizado com find
- Valor em estoque é calculado
- Categorias repetidas são removidas
- Promise.all reúne os dois resultados

Exercite as falhas

```
npm run produto-ausente
npm run id-invalido
del .env
npm run diagnostico
```

Evidências esperadas

- produto-ausente: produto 999 não encontrado e código 1
- id-invalido: identificador inteiro exigido
- diagnostico: nomes obrigatórios ausentes
- Use echo %ERRORLEVEL% para confirmar o código final

Atividades

08

Cinco aplicações independentes para consolidar a aula.

Atividades 01 a 05

- 01: atualizar estoque com spread
- 02: filtrar materiais disponíveis
- 03: criar uma Promise para pedido
- 04: tratar rejeição de sensor
- 05: reunir estoque, reservas e preço com Promise.all

Protocolo de trabalho

- Leia o README da atividade
- Execute primeiro o estado PENDENTE
- Implemente somente o TODO
- Execute o cenário de sucesso
- Execute o cenário de falha
- Confirme o resultado com echo %ERRORLEVEL%

Desafio de fechamento

09

Explique cada resultado antes de executar.

Preveja map, filter e find

```
node -e "const a=[1,2,3]; console.log(a.map(x=>x*2))"
node -e "const a=[1,2,3]; console.log(a.filter(x=>x>3))"
node -e "const a=[1,2,3]; console.log(a.find(x=>x>3))"
```

Quais são os três resultados?

Responda antes de executar ou avançar

Escolha a sequência correta.

A) [2,4,6] | [] | undefined

B) [2,4,6] | undefined | []

C) [1,2,3] | [] | 3

Checklist de saída

- const e let são escolhidos pela necessidade de reatribuição
- Spread cria uma nova versão do objeto
- map, filter e find entregam resultados diferentes
- async/await consome Promises sem bloquear o Event Loop
- Promise.all coordena operações independentes
- O .env local foi removido ao terminar

Síntese da aula

- Transformações podem preservar a entrada
- Promises representam resultados futuros
- async/await organiza sucesso e falha
- Concorrência sobrepõe esperas independentes
- Promise.all não cria paralelismo de CPU

Referências oficiais

Documentação usada na Aula 02

MDN: const, desestruturação e spread	developer.mozilla.org/docs/Web/JavaScript/Reference
MDN: Array	developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Array
MDN: Promise	developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Promise
MDN: async function	developer.mozilla.org/docs/Web/JavaScript/Reference/Statements/async_function
Node.js: timers	nodejs.org/api/timers.html